23 Security Issue Implementation Prompts

Full-stack Web App / SaaS Security Hardening

Ready-to-paste prompts for an AI coding agent or development team. Each prompt covers the intended security control, implementation direction, and acceptance criteria.

Important: These prompts are designed for defensive security work on an application you own or are authorized to test. Where the handwriting in the source notes was ambiguous, the requirement has been normalized into a practical, production-oriented security task.

How to use this document

Run the prompts one issue at a time rather than asking an AI coding agent to blindly change the entire application in one pass. For every issue, require the agent to inspect the existing architecture first, make the smallest safe change, run tests, and report exactly what changed.

- Always inspect the existing frontend, backend, database schema, API routes, authentication, deployment configuration, and environment-variable conventions before editing.
- Do not invent infrastructure or replace working authentication/database systems without evidence that the current implementation requires it.
- Never print real secrets, credentials, tokens, private keys, or sensitive user data in the AI response or generated report.
- After each fix, run unit/integration/security tests and verify that legitimate functionality still works.
- For production, use managed secrets, least privilege, centralized logging/monitoring, backups, and staged deployment.

Issue index

1. Server-Side Template Injection (SSTI)
2. ReDoS / DoS Protection
3. Low-Rate DoS (LDoS)
4. Secret-Key Leakage — Move Secrets to Environment/Secret Management
5. SQL / NoSQL Injection + Oversized JSON
6. Clipboard Attack Protection
7. Replay Attack Protection
8. Rate Limiting on All Relevant Endpoints
9. Full Codebase Scan for Hardcoded API Keys, Tokens, Passwords
10. Environment Variables & Frontend Secret Boundary
11. Input Sanitization & Safe Output Handling
12. Password Protection & Credential Security
13. Full Security Audit & Vulnerability Report
14. Authentication, Authorization & Git/Domain Exposure
15. Broken Access Control / IDOR
16. Web Input / URL / Request Validation
17. Database & Backend Security Hardening
18. Secure Login Page & Authentication Flow
19. Caching Management & Sensitive Data Protection
20. API & AI Model Subscription / Key Management
21. SQL, Database Fixes & Ongoing DB Management
22. SEO, Google Analytics & Sentry Privacy/Security
23. Master Security Hardening — Manage Everything End-to-End

Issue 1: Server-Side Template Injection (SSTI)

Copy/paste prompt

Inspect the entire application for server-side template injection. Identify every place where user-controlled data can reach a server-side template engine, dynamic rendering function, expression evaluator, email template, document generator, or similar renderer. Remove unsafe string-to-template evaluation and replace it with fixed templates plus explicitly mapped variables. Never allow user input to become template code. Add strict allowlists for template names and variables, contextual output encoding, and server-side validation. If a template engine supports sandboxing, enable it, but do not treat sandboxing as the only defense. Add security tests for common SSTI payload patterns and verify that they are rendered as inert text. Review frontend, backend, API routes, background jobs, admin tools, and database-driven templates.

Acceptance criteria

- No user-controlled value is evaluated as template/expression code.
- Templates are selected from an allowlist and variables are explicitly mapped.
- Security tests cover server-side rendering endpoints and fail safely.
- Errors do not expose template internals, stack traces, secrets, or filesystem paths.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 2: ReDoS / DoS Protection

Copy/paste prompt

Audit the application for regular expressions, parsers, uploads, and endpoints that can be abused for CPU, memory, connection, or worker exhaustion. Find catastrophic/backtracking regexes and replace them with safe patterns, bounded parsers, or non-regex validation where possible. Enforce request size, header size, upload size, JSON depth, array length, string length, pagination limits, and processing timeouts. Add concurrency protection and cancellation/timeouts for expensive operations. Apply appropriate rate limits at the edge and API layer. Do not use a single blanket limit for every route; choose limits based on endpoint sensitivity and cost. Add automated tests for oversized and pathological inputs.

Acceptance criteria

- No catastrophic regex patterns remain in security-sensitive paths.
- Requests and uploads have explicit size/time/resource limits.
- Expensive operations cannot consume unbounded workers or memory.
- Rate limiting and timeouts return controlled errors without crashing the service.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 3: Low-Rate DoS (LDoS)

Copy/paste prompt

Protect the application against low-and-slow or low-rate denial-of-service attacks that keep connections, sessions, queues, or expensive operations occupied without generating a high request count. Audit HTTP connection handling, streaming endpoints, file uploads, WebSockets, long polling, database connections, background jobs, and third-party API calls. Configure connection, header, body, idle, read, write, and total request timeouts where supported. Limit concurrent connections and per-user/IP resource consumption, with proxy/load-balancer protections where available. Ensure slow clients cannot monopolize workers. Add monitoring for unusual connection duration, concurrency, queue depth, and resource saturation. Keep legitimate long-running workflows functional through explicit asynchronous job patterns rather than indefinite requests.

Acceptance criteria

- Slow connections have bounded timeouts.
- Per-client and global concurrency/resource limits are enforced.
- Long jobs use asynchronous processing where appropriate.
- Monitoring can detect abnormal connection duration and resource exhaustion.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 4: Secret-Key Leakage — Move Secrets to Environment/Secret Management

Copy/paste prompt

Perform a complete secrets audit. Find API keys, database credentials, JWT secrets, OAuth secrets, webhook secrets, encryption keys, service-account credentials, private keys, passwords, and other sensitive configuration in source code, frontend bundles, public assets, logs, documentation, example files, CI configuration, Git history, and database records. Move server-only secrets into a secure environment/secret-management system. Never expose server secrets through frontend environment variables, public configuration, browser code, HTML, API responses, or source maps. Use separate secrets per environment, rotate any credential that may already have been exposed, and update .gitignore and CI secret handling. Provide a safe .env.example containing placeholders only.

Acceptance criteria

- No real secrets exist in tracked source or frontend bundles.
- Server-only secrets are accessed only on the server.
- Potentially exposed credentials are rotated.
- CI/CD and local development use documented secret injection without committing values.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 5: SQL / NoSQL Injection + Oversized JSON

Copy/paste prompt

Audit every database interaction for SQL injection and NoSQL injection. Replace string-concatenated queries and unsafe dynamic filters with parameterized queries, prepared statements, query builders, or safe ORM APIs. For NoSQL, explicitly validate and allowlist accepted fields/operators and prevent user input from becoming query operators or executable expressions. Validate request bodies against strict schemas. Reject excessively large JSON bodies, deeply nested objects, huge arrays, and unexpectedly large strings before expensive processing. Apply server-side authorization independently of validation. Add negative tests for injection-like input and ensure the application returns safe validation errors without leaking database details. Log security events without logging secrets or sensitive payloads.

Acceptance criteria

- All database queries are parameterized or use safe query APIs.
- User input cannot introduce arbitrary operators or query structure.
- JSON/body limits are enforced before expensive processing.
- Database errors are sanitized and do not reveal SQL, stack traces, credentials, or schema internals.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 6: Clipboard Attack Protection

Copy/paste prompt

Audit all copy, paste, clipboard, and contenteditable functionality. Treat clipboard data as untrusted input. Prevent pasted HTML, scripts, event handlers, dangerous URLs, or rich content from becoming executable markup. Prefer plain-text paste for fields that do not need rich formatting. If rich text is required, sanitize it with a maintained allowlist-based HTML sanitizer and remove scripts, event handlers, dangerous protocols, and unsafe embedded content. Review copy-to-clipboard functionality for sensitive data: do not silently copy passwords, tokens, API keys, or private information. Ensure clipboard permissions are requested only when needed and use secure browser APIs. Add tests for malicious pasted HTML and javascript/data URLs.

Acceptance criteria

- Pasted content is sanitized or converted to plain text.
- No clipboard value can introduce executable HTML/JavaScript.
- Sensitive secrets are not automatically copied or exposed.
- Clipboard permissions and APIs are used minimally and safely.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 7: Replay Attack Protection

Copy/paste prompt

Identify endpoints where an attacker could capture and resend a valid request, token, payment instruction, webhook, or state-changing action. For high-risk operations, implement replay resistance using short-lived tokens, unique request IDs/nonces, timestamps with a bounded clock-skew window, and server-side idempotency records where appropriate. For webhooks, require signed requests and verify signature, timestamp, and event/request ID before processing. Ensure access tokens have appropriate lifetimes and are revoked/rotated when necessary. Do not rely on client-side timestamps alone. Make state-changing operations idempotent so retries do not duplicate side effects. Add tests that resend the same valid request and verify the second attempt is rejected or safely treated as an idempotent retry.

Acceptance criteria

- Repeated high-risk requests cannot create duplicate side effects.
- Replay-sensitive requests have server-validated freshness/uniqueness.
- Webhook signatures and timestamps are verified before processing.
- Idempotency keys are stored safely with bounded retention.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 8: Rate Limiting on All Relevant Endpoints

Copy/paste prompt

Implement centralized, server-side rate limiting for authentication, password reset, OTP, signup, API keys, search, uploads, expensive AI operations, payment/subscription actions, and other abuse-prone endpoints. Use route-specific limits rather than one global value. For example, authentication attempts may use a stricter policy such as 5 failures per 15 minutes per account/IP combination, with progressive backoff; choose production values based on legitimate traffic and risk. Use a shared store such as Redis when multiple server instances are deployed. Return HTTP 429 with a safe Retry-After value. Prevent attackers from bypassing limits through alternate routes, header spoofing, or multiple identifiers. Do not rate-limit trusted internal calls blindly.

Acceptance criteria

- Rate limits are enforced on the server and work across multiple instances.
- Sensitive routes have stricter, adaptive controls.
- 429 responses are consistent and do not leak account existence.
- Monitoring shows rejected requests and identifies abuse patterns.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 9: Full Codebase Scan for Hardcoded API Keys, Tokens, Passwords

Copy/paste prompt

Scan the complete repository, including frontend, backend, scripts, tests, configuration, CI/CD files, Docker files, documentation, generated files, and Git history where available, for hardcoded credentials and sensitive tokens. Use secret-scanning tooling plus manual review. Search for common credential patterns and provider-specific keys, but do not print discovered secrets into logs or the final report. For every finding, identify the file and type of secret without reproducing the value. Move secrets to secure configuration, rotate exposed credentials, remove them from source control, and if necessary clean Git history using an approved process. Add pre-commit and CI secret scanning so future commits are blocked or flagged.

Acceptance criteria

- No live credential remains hardcoded.
- Secret scanning runs in CI and, ideally, pre-commit hooks.
- Exposed credentials are rotated, not merely deleted from the current file.
- Reports redact secret values.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 10: Environment Variables & Frontend Secret Boundary

Copy/paste prompt

Move all sensitive backend configuration to server-side environment variables or a managed secret store. Create a clear distinction between public frontend configuration and private server configuration. Audit framework-specific public prefixes (for example, variables intentionally exposed to the browser) and ensure no secret is placed in them. Verify that API keys, database URLs, private signing keys, service credentials, and internal endpoints are never bundled into browser JavaScript. Ensure .env files containing real secrets are ignored by Git and are not included in Docker images or build artifacts. Provide .env.example with dummy values only. Add a build-time check that fails when known secret patterns are detected in client bundles.

Acceptance criteria

- Frontend builds contain only intentionally public configuration.
- Private environment variables are accessible only to trusted server-side code.
- Real .env files are excluded from Git/build artifacts.
- CI checks prevent accidental exposure.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 11: Input Sanitization & Safe Output Handling

Copy/paste prompt

Create a centralized input-validation strategy for every user-controlled input: body, query, path parameters, headers, cookies, file names, uploaded files, metadata, and third-party webhook data. Validate with strict schemas, type checks, length limits, enumerations, and business rules. Sanitize only where appropriate; do not use generic string replacement as a substitute for contextual security controls. For HTML output, use contextual escaping and a maintained sanitizer for allowed rich text. For SQL/NoSQL, use parameterized query APIs. For URLs, allowlist protocols and hosts where relevant. Reject malformed or unexpected input by default. Add tests for XSS, injection, path traversal, malformed JSON, dangerous URLs, and oversized values.

Acceptance criteria

- Every external input has server-side validation.
- Validation occurs before database writes or expensive processing.
- Output is encoded/sanitized according to its context.
- Malformed input produces safe 4xx responses without stack traces.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 12: Password Protection & Credential Security

Copy/paste prompt

Harden authentication and password handling end-to-end. Never store plaintext or reversible passwords. Use a modern password hashing algorithm such as Argon2id or a properly configured bcrypt/scrypt implementation. Enforce secure password reset flows with short-lived, single-use tokens. Protect login and reset endpoints with rate limiting and progressive delays. Prevent account enumeration through consistent responses where appropriate. Use secure, HttpOnly, SameSite cookies for session credentials when cookies are used, and protect state-changing requests against CSRF as needed. Add MFA support where appropriate for privileged accounts. Never log passwords, reset tokens, session tokens, or authentication headers.

Acceptance criteria

- Passwords are strongly hashed and never retrievable.
- Reset tokens are single-use, short-lived, and stored/handled safely.
- Authentication endpoints are rate-limited.
- Session cookies/tokens use secure flags and appropriate expiration/revocation.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 13: Full Security Audit & Vulnerability Report

Copy/paste prompt

Run a full security audit of the application before making assumptions about what is secure. Inspect frontend, backend, database, authentication, authorization, API routes, third-party integrations, file uploads, storage, infrastructure configuration, CI/CD, dependencies, logging, monitoring, and production configuration. Check for OWASP Top 10 risks and common cloud/web misconfigurations. Use automated dependency and secret scanning where available, then manually review high-risk flows. Produce a prioritized report with severity, affected component, evidence, impact, recommended remediation, and verification status. Do not include exploit payloads that would cause destructive behavior, and never expose real secrets in the report.

Acceptance criteria

- A documented inventory of findings exists with severity and remediation status.
- Critical/high-risk issues are addressed or explicitly accepted with rationale.
- Dependency, secret, authentication, authorization, and configuration checks are included.
- Security regression tests are added for fixed vulnerabilities.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 14: Authentication, Authorization & Git/Domain Exposure

Copy/paste prompt

Audit authentication and authorization across the entire application, including admin routes, API routes, server actions, storage, database access, and frontend route guards. Do not rely on frontend checks for security. Enforce permissions server-side on every protected resource and use least-privilege roles. Review Git repository visibility, deployment configuration, preview environments, custom domains, DNS, CORS, and public endpoints so private source/configuration is not unintentionally exposed. Remove unnecessary public endpoints and debug routes. Ensure production authentication is enforced consistently across the main domain and alternate/preview domains.

Acceptance criteria

- Unauthorized users cannot access protected API/data by calling endpoints directly.
- Role checks are enforced server-side.
- Private repositories/configuration are not publicly exposed.
- Preview/staging domains are protected and cannot bypass production authorization.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 15: Broken Access Control / IDOR

Copy/paste prompt

Perform an object-level authorization audit for every endpoint that accepts an ID, UUID, filename, email, account ID, organization ID, invoice ID, order ID, subscription ID, or similar identifier. Prevent Insecure Direct Object Reference (IDOR) by checking that the authenticated user is authorized to access the specific object and tenant. Never trust IDs supplied by the client. Implement centralized authorization helpers/policies and enforce them before reading or modifying data. Test horizontal privilege escalation (user A accessing user B data), vertical privilege escalation (normal user accessing admin resources), and cross-tenant access. Ensure list/search endpoints also apply authorization filters.

Acceptance criteria

- Every object read/write has server-side ownership or permission checks.
- Cross-user and cross-tenant access tests fail safely.
- Changing an ID in a request cannot expose another user's data.
- Authorization is applied consistently to read, update, delete, export, and download operations.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 16: Web Input / URL / Request Validation

Copy/paste prompt

Audit all web-facing request parameters, URLs, redirects, headers, cookies, forms, and API payloads. Implement strict allowlist-based validation for values with a known format and reject unexpected fields where practical. Protect against open redirects, malicious URLs, header injection, path traversal, unsafe file names, prototype-pollution-style object merging, and malformed requests. Normalize input before validation where required, but preserve security boundaries. Do not trust client-side validation. Add centralized schemas for each API route and ensure validation errors are safe and consistent.

Acceptance criteria

- External input is validated server-side with route-specific schemas.
- Open redirects and dangerous URL schemes are blocked.
- Unexpected object properties cannot alter security-sensitive server behavior.
- Path/file inputs are normalized and constrained to intended resources.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 17: Database & Backend Security Hardening

Copy/paste prompt

Harden the database and backend architecture. Enforce least-privilege database accounts, separate application and migration/admin credentials, private network access where possible, encrypted connections, encrypted backups, and secure secret storage. Disable unnecessary database extensions, users, ports, and public access. Add connection pooling limits, query timeouts, transaction boundaries, and safe error handling. Ensure the backend never trusts the database as an authorization layer alone. Review ORM configuration, migrations, indexes, backups, logs, and data retention. Prevent sensitive data from being returned unnecessarily by APIs.

Acceptance criteria

- Production databases are not unnecessarily public.
- Application DB credentials have only required permissions.
- Backups and connections are protected.
- Backend APIs return minimum necessary data and sanitize database errors.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 18: Secure Login Page & Authentication Flow

Copy/paste prompt

Harden the complete login and registration flow across frontend and backend. Support secure email/password authentication and, if Google login is enabled, validate OAuth/OIDC tokens on the server and enforce correct redirect URI and state/nonce checks. Do not implement authentication solely in frontend code. Use secure session management, CSRF protection where applicable, account lockout/progressive throttling, email verification, password reset, and safe error messages. Protect tokens in transit and storage. Ensure authenticated pages cannot be accessed after logout and that session invalidation works server-side.

Acceptance criteria

- Login and registration are enforced by the backend.
- OAuth state/nonce and redirect URIs are validated.
- Sessions are securely created, refreshed, revoked, and expired.
- Logout invalidates the session and protected APIs reject the old credential.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 19: Caching Management & Sensitive Data Protection

Copy/paste prompt

Audit all application, CDN, browser, API, Redis, and server-side caches. Prevent personalized or sensitive responses from being cached and served to another user. Set explicit Cache-Control/Vary policies for authentication-dependent responses. Review cache keys so tenant/user identity cannot collide. Avoid storing secrets, session tokens, private profile data, or sensitive API responses in shared caches. Add cache invalidation for permission changes, logout, password reset, subscription changes, and sensitive data updates. Ensure Redis or other cache services are authenticated, network-restricted, and not publicly exposed.

Acceptance criteria

- Private responses are not stored in shared/public caches.
- Cache keys include the necessary security context.
- Logout/password reset/permission changes invalidate relevant cached data.
- Cache infrastructure is access-controlled.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 20: API & AI Model Subscription / Key Management

Copy/paste prompt

Audit all external API and AI-model integrations. Keep provider API keys server-side and never expose them in frontend code. Store provider credentials in secure environment/secret management, rotate them periodically, and use separate keys per environment/provider where supported. Apply provider-specific spend limits, quotas, timeouts, retries with exponential backoff, and per-user usage controls. Validate and constrain user input before sending it to external models. Do not send unnecessary personal or confidential data to third-party providers. Log usage metadata without logging secrets or sensitive prompts unless explicitly required and protected.

Acceptance criteria

- Provider keys are never exposed to browsers.
- Usage limits and spend controls exist for expensive APIs.
- Timeouts/retries cannot create runaway costs.
- Third-party data sharing is minimized and documented.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 21: SQL, Database Fixes & Ongoing DB Management

Copy/paste prompt

Review the SQL layer and database operations for correctness, security, and maintainability. Remove raw SQL built from user input unless it is safely parameterized. Validate migrations and schema constraints, enforce foreign keys and uniqueness where appropriate, and add indexes for safe predictable queries. Review transaction handling and race conditions around payments, balances, subscriptions, inventory, or other state changes. Add database-level constraints for critical invariants and application-level authorization before queries. Establish a safe backup, migration, rollback, and monitoring process. Test concurrent requests for duplicate or inconsistent state.

Acceptance criteria

- Critical queries are parameterized.
- Database constraints protect important invariants.
- Transactions prevent partial updates in critical workflows.
- Backups/migrations are tested and reversible.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 22: SEO, Google Analytics & Sentry Privacy/Security

Copy/paste prompt

Audit SEO, analytics, and error-monitoring integrations for privacy and security. Ensure Google Analytics and Sentry receive only the minimum data required and never receive passwords, access tokens, authorization headers, payment data, private messages, or unnecessary personal information. Configure Sentry event scrubbing/redaction and review source-map handling so production source maps do not expose sensitive implementation details publicly. Add appropriate consent/privacy controls where legally required. Review robots.txt, sitemap, canonical URLs, metadata, and indexing rules without accidentally exposing private/admin pages. Ensure analytics IDs and public configuration are not confused with secret credentials.

Acceptance criteria

- Sensitive fields are scrubbed before analytics/error reporting.
- Sentry does not capture authentication secrets.
- Private/admin routes are not unintentionally indexed.
- Public analytics identifiers are separated from secret credentials.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.

Issue 23: Master Security Hardening — Manage Everything End-to-End

Copy/paste prompt

Perform a final end-to-end security hardening pass across the entire application. Inspect frontend, backend, database, authentication, authorization, API routes, file uploads, storage, caching, secrets, dependencies, AI/API integrations, payments/subscriptions, logging, monitoring, analytics, deployment, domains, Git, CI/CD, backups, and infrastructure. Treat every client-controlled value as untrusted and every privileged operation as server-authorized. Remove hardcoded secrets, enforce secure headers, HTTPS, CORS restrictions, CSRF protections where applicable, secure cookies, input validation, output encoding, rate limits, timeouts, audit logging, dependency updates, and least privilege. Run automated tests and security scans, then manually test critical flows. Do not break legitimate product functionality; document any security control that requires a product trade-off. Produce a final security report showing each issue as Fixed, Not Fixed, or Accepted Risk with evidence and next steps.

Acceptance criteria

- All 23 checklist areas have an owner/status and verification evidence.
- Critical/high findings are remediated or formally accepted.
- Security controls are implemented server-side, not only in the UI.
- CI/CD includes repeatable secret, dependency, and security checks.
- Monitoring and incident-response information is available for production.

Implementation note: Before changing code, have the agent identify the affected files/routes/services and explain the planned fix. Require tests and a concise change report. Avoid exposing real secrets in logs or output.